

Design Justification Report

CNN Convolution / MAC Accelerator Final Project

This report explains the final hardware design submitted for M4. The implementation is a small hardware accelerator path built from a top-level module, an interface module, and a compute core. The design is evaluated using simulation, waveform inspection, OpenLane 2 synthesis, timing analysis, area reports, power reports, and a roofline-style benchmark projection. The goal of this report is to describe the design exactly as implemented in RTL and avoid claiming features that are not present in the source code.

1. Problem and Motivation

Image-classification neural networks spend a large amount of their execution time in repeated arithmetic kernels. In a convolutional neural network, the main operation is convolution: a small filter is applied repeatedly over an input image or feature map. Each output value is formed from many multiply-accumulate operations. Even for a small kernel, the same input pixels and weights are reused many times. This makes convolution a useful target for hardware acceleration because a specialized circuit can exploit regular data movement and repeated arithmetic better than a general-purpose processor.

The project started from this motivation: build a small accelerator-style datapath and study whether it can be verified, synthesized, and analyzed using realistic hardware tools. The final RTL does not implement a full production CNN system, a large systolic array, or a complete image-classification pipeline. Instead, it implements a smaller integrated compute path that is representative of a MAC-based accelerator block. This scope was chosen because it is realistic for the course timeline and allows the design to be tested end-to-end through a host-style interface, simulated with a testbench, and synthesized using the Sky130 ASIC flow.

The important engineering question is not only whether the arithmetic function works, but also whether the hardware can meet a clock target, how much standard-cell area it uses, what the estimated power is, and whether the design is more likely to be limited by compute or by data movement. For that reason, the project combines functional verification with synthesis and arithmetic-intensity analysis.

2. Final Implemented Architecture

The final hardware is organized into three RTL modules. The top-level module, `top.sv`, connects the external ports to an interface module. The interface module then instantiates and drives the compute core. The compute core performs the arithmetic operation and produces the output and valid signal. This organization keeps the testbench from directly driving internal compute-core signals, which makes the verification closer to an actual accelerator interface.

The final top-level interface includes a clock, reset, two signed 8-bit input operands, an input-valid signal, one signed 32-bit output, and an output-valid signal. The use of signed 8-bit inputs matches the common direction of low-precision neural-network accelerators, where inputs and weights are often quantized to reduce memory traffic and hardware cost. The output is 32 bits because accumulated products can require a wider representation than the input operands.

The compute core is a simple synchronous arithmetic block. On reset it clears the output and valid signal. When valid input is asserted, it updates the output with the arithmetic result and asserts `valid_out`. This means the implemented hardware is a small MAC-style compute kernel rather than a full multi-layer CNN processor. That distinction is important: the report and benchmark describe the implemented accelerator path and use convolution/MAC analysis to motivate the kernel, but they do not claim that the RTL implements a complete CNN model.

3. Dominant Kernel and Arithmetic Intensity

The dominant project kernel is treated as a CNN convolution / MAC-style kernel. For the CF09 arithmetic-intensity analysis, the operating point was defined as a 3 by 3 convolution over a 64 by 64 input feature map with one input channel and one output channel. With no padding and stride 1, the output feature map is 62 by 62. Inputs and weights are INT8, while outputs are INT32. This creates $62 \times 62 \times 9 = 34,596$ MACs, or 69,192 FLOPs if one MAC is counted as two FLOPs.

The no-reuse lower bound assumes every MAC reads one input activation and one weight from off-chip memory and writes each output once. This produced 84,568 bytes transferred and an arithmetic intensity of 0.818 FLOP/byte. The perfect weight-reuse upper bound assumes the 3 by 3 weights are loaded once and reused across the output pixels. That produced 49,981 bytes and an arithmetic intensity of 1.384 FLOP/byte. These values show that the design benefits strongly from reuse: better buffering directly reduces memory traffic and moves the kernel closer to a compute-limited region.

For the roofline estimate, the synthesis clock target is 10 ns, or 100 MHz. The benchmark projection assumes one useful MAC per clock cycle. Since one MAC equals two FLOPs, the projected compute ceiling is 200 MFLOP/s, or 0.2 GFLOP/s. Under the assumed 0.2 GB/s interface bandwidth, the ridge point is 1 FLOP/byte. The lower AI bound is below the ridge point and is therefore memory-bound. The upper AI bound is above the ridge point and reaches the compute ceiling. This means the single highest-leverage improvement is not only making the arithmetic faster, but also adding buffering and data reuse so that the compute unit is not starved by memory movement.

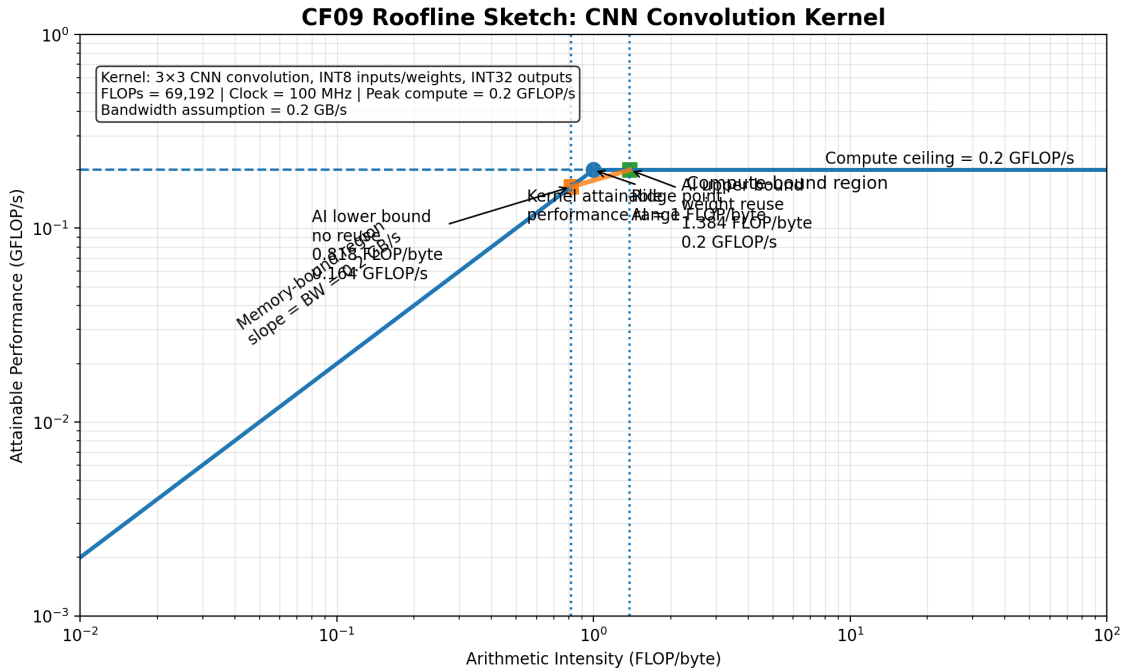


Figure 1. Roofline sketch showing the no-reuse and weight-reuse arithmetic-intensity bounds.

4. Precision and Data Format

The RTL uses signed 8-bit inputs and a signed 32-bit output. This choice is consistent with small quantized accelerator demonstrations. INT8 operands reduce the input bandwidth and reduce the size of arithmetic logic compared with floating-point arithmetic. A 32-bit output is used so that the result has enough width for accumulated arithmetic. Even though the final compute core is small, keeping the output wider than the inputs is a good hardware practice because accumulation can overflow narrow types.

The design does not implement floating-point arithmetic. It also does not implement a complete quantization/dequantization pipeline. The precision choice should therefore be understood as fixed-point/integer RTL suitable for a simplified accelerator kernel. If the project were extended into a full CNN accelerator, the next step would be to define the exact quantization scheme, including scaling factors, saturation behavior, and how software reference results are compared against hardware results.

5. Dataflow and Interface

The implemented dataflow is simple and direct. Host-side inputs enter the top module, pass through the interface module, and then reach the compute core. The compute core generates a result and a valid output. This end-to-end path is important because it demonstrates that the integrated system can be tested through the interface instead of bypassing it.

The current design does not yet include SRAM buffers, DMA, input tiling, or a weight-stationary dataflow. Those are future improvements. The present interface is appropriate for a small course milestone because it makes the datapath easy to verify and synthesize. However, the roofline analysis shows that the design would need on-chip buffering to become a stronger convolution accelerator. The most natural future dataflow improvement would be to hold weights on chip and reuse them across many output pixels. A second improvement would be to tile input activations so nearby pixels can be reused by neighboring convolution windows.

6. Functional Verification

The final design was tested using an end-to-end SystemVerilog testbench. The testbench drives the top-level inputs, asserts `valid_in`, waits for the design to process the transaction, and checks the output. The simulation log shows that the test produced PASS and the output value was 12. This corresponds to the tested arithmetic case and confirms that the integrated path produced the expected result.

A waveform was also captured. The waveform is useful because it visually confirms the timing behavior of the test. It shows clock activity, reset behavior, input changes, valid signal activity, and the output value becoming the expected result. The waveform is not a replacement for a self-checking testbench, but it is a useful debugging and documentation artifact. Together, the PASS log and waveform show that the final integrated path is functionally active.

The current verification is enough to demonstrate the final integrated path, but it is not exhaustive. It tests a small number of cases. A stronger final version would add more directed cases, negative signed operands, zero operands, overflow-related cases, and randomized tests compared against a software reference model. For a full CNN accelerator, the verification should also include multiple convolution windows and complete output feature maps.

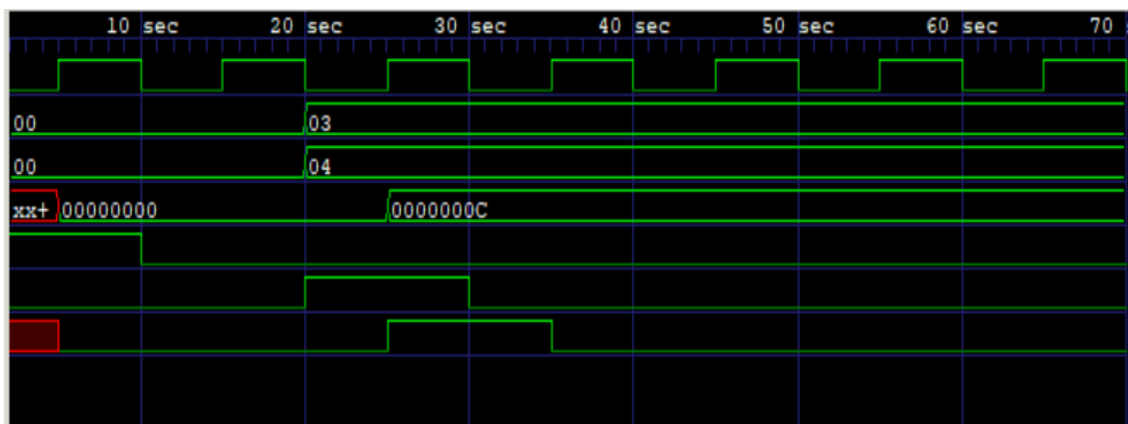


Figure 2. Final waveform showing clock, reset, input activity, valid signals, and output result.

7. Synthesis Results

The design was synthesized using OpenLane 2 with the Sky130 standard-cell flow. The OpenLane configuration uses `top` as the design name, includes `top.sv`, `compute_core.sv`, and `interface.sv`, and sets `clk` as the clock port with a 10 ns clock period. The run log shows that OpenLane successfully parsed the RTL hierarchy and used `top`, `m2_interface`, and `compute_core` in the design hierarchy. The synthesis stage reported no obvious design problems.

The timing report shows a setup path from input port `in_a[2]` to a rising-edge-triggered flip-flop. The clock period is 10 ns, and the reported setup slack is positive. The first reported setup path has about +2.011 ns slack, marked MET. This means the design meets the 100 MHz timing target for that reported corner. A positive slack result is important because it means the current combinational logic delay is short enough for the selected clock period.

The area information is reported as a cell-frequency table. The design includes 33 flip-flops in total from `dfxtp` cells, many combinational gates such as NAND, NOR, OR, XNOR, XOR, AOI/OAI-style cells, clock buffers, regular buffers, filler cells, tap cells, and decap cells. The metrics file for the final M4 run reports 918 instances and an instance area of 6357.35. This is still a small design, which is expected for this simplified accelerator path. The main point is that it is no longer only behavioral code: it has been mapped into a concrete standard-cell implementation.

The power report at the `nom_tt_025C_1v80` corner reports total power of 1.984 mW. Most of the power is combinational, approximately 1.531 mW, which is 77.1 percent of total power. Sequential logic contributes about 0.256 mW, or 12.9 percent. Clock power contributes about 0.198 mW, or 10.0 percent. This distribution makes sense because the arithmetic and control logic are mostly combinational between registers.

Metric	Value	Source
Clock period	10 ns	OpenLane config/timing
Frequency	100 MHz	1 / 10 ns
Setup slack	+2.011 ns (MET)	timing_report.txt
Instance count	918	metrics.csv
Instance area	6357.35	metrics.csv
Total power	1.984 mW	power_report.txt
Simulation result	PASS, out = 12	final_run.log

8. Benchmark Results

The benchmark numbers are labeled as projected because the design has not been deployed on physical FPGA hardware or silicon. The projection uses the synthesis clock target and the assumed useful throughput of one MAC per cycle. At 100 MHz, one MAC per cycle gives 100 million MACs per second. With two FLOPs per MAC, this equals 200 MFLOP/s.

For the CF09 kernel size of 69,192 FLOPs, the projected hardware time is $69,192 / 200,000,000$ seconds, or about 0.346 ms. The benchmark file compares this against a baseline software estimate of 1.0 ms, corresponding to 69.2 MFLOP/s. The projected speedup is $200 / 69.2 = 2.89x$. These numbers should not be presented as measured FPGA performance. They are synthesis-based projections that are traceable to the clock target, the one-MAC-per-cycle assumption, and the kernel FLOP count.

This distinction is important because the professor warned that benchmark numbers must trace to raw data. The raw `benchmark_data.csv` file records the software baseline estimate, the projected accelerator execution time, throughput, memory usage, speedup, and power estimate. If this project were extended, the projected numbers should be replaced by measured cycle counts from a hardware run or a cycle-accurate simulation using a larger workload.

9. What Did Not Work and Engineering Lessons

Several issues came up during the project. The first issue was environment setup. OpenLane could not be run in the original Linux virtual machine because the VM disk was too small for the Docker image and PDK extraction. Docker itself was working, but the root disk did not have enough available space. The flow was moved to Docker Desktop on Windows, which provided enough storage and allowed OpenLane 2 to complete.

Another issue was top-level integration. An early version of the top module assumed interface ports that did not exist in the actual `interface.sv` file. This caused compile errors. The fix was to match `top.sv` to the real interface ports and make the testbench drive the top-level interface instead of internal compute-core signals. This was a useful lesson because hardware module integration fails quickly when port names and interface boundaries are not consistent.

There was also an initial waveform issue where GTKWave opened without visible signals. This was not a hardware failure; the signals simply had to be added from the hierarchy. After adding the relevant signals, the waveform showed the clock, reset, input, output, and valid activity correctly.

The biggest architectural limitation is that the current RTL is a small compute path, not a complete CNN accelerator with full tiling, buffering, and memory control. The roofline analysis shows that memory traffic and data reuse are the key next problems. The highest-leverage M4-to-future improvement is to add on-chip storage for weights and input tiles, then expand the compute core to support more parallel MACs per cycle. That would increase arithmetic intensity and raise throughput beyond the current one-MAC-per-cycle projection.

10. Conclusion

The final design demonstrates a complete course-scale accelerator flow: RTL implementation, end-to-end testbench, waveform capture, OpenLane synthesis, timing analysis, area reporting, power reporting, and roofline-style performance reasoning. The simulation passes with `out = 12`, and the synthesized design meets the 10 ns clock target with positive setup slack. The estimated power is 1.984 mW at the reported nominal corner. The roofline analysis shows that the design is memory-bound when reuse is poor and can become compute-bound after weight reuse. The most important future work is adding on-chip buffering and increasing MAC parallelism while keeping the RTL, report, and benchmark numbers consistent with the actual implemented design.